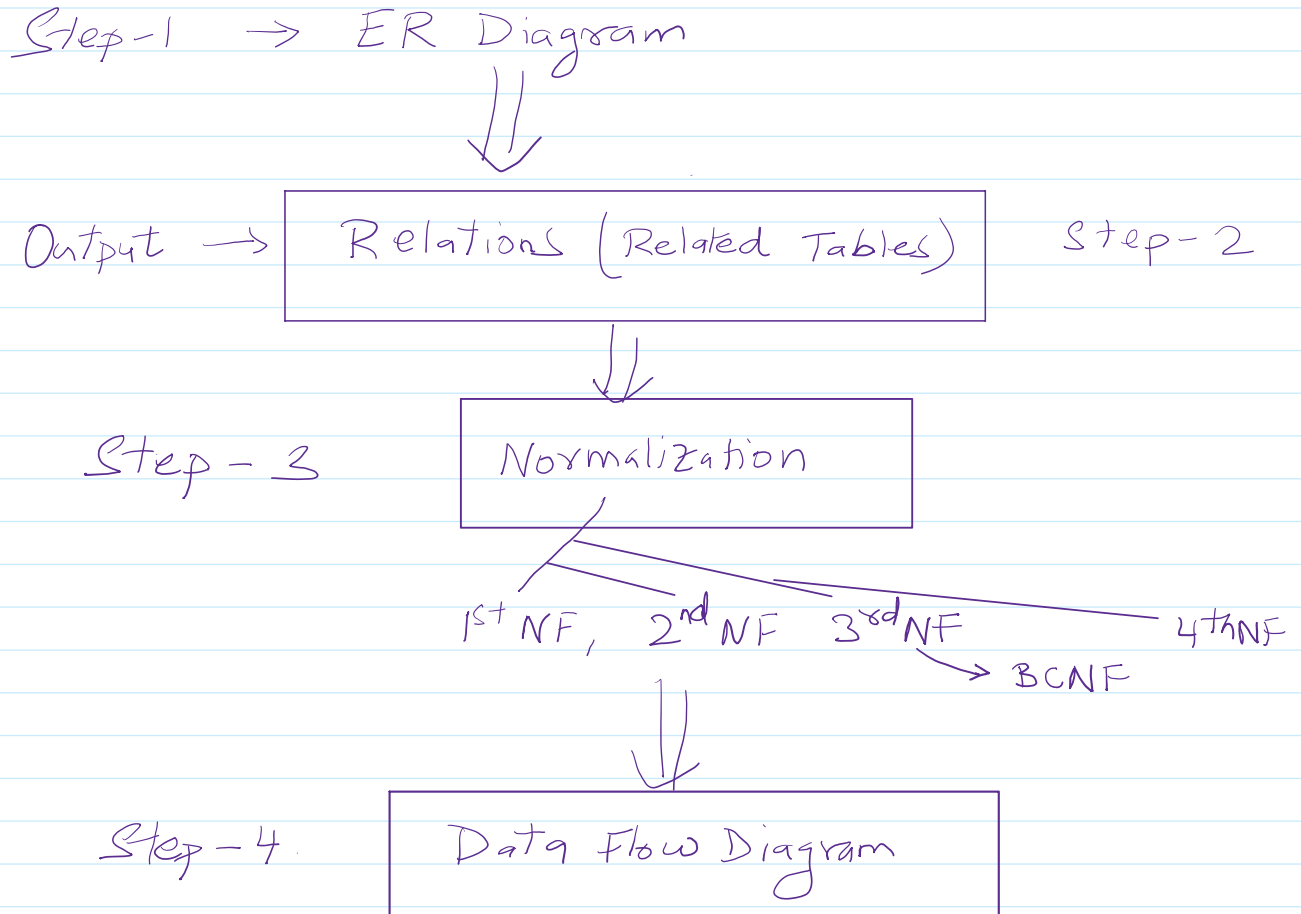


ER Diagrams

25 September 2026 19:06



ER Diagram

An ER diagram (Entity-Relationship Diagram) is a graphical representation of the data, entities, attributes, and relationships in a database.

It is mainly used during the database design phase to understand how data will be organized before creating tables in a DBMS such as Mysql, Oracle. Postgresql or SQL server, etc.

Why do we need an ER diagram?

Suppose we want to create a database for a college management system.

The college needs to store information about:

- Students
- Teachers
- Courses
- Departments
- Subjects

- Enrollments

Before creating database tables we need to understand:

- What information should be stored?
- What are important objects?
- What properties does each object have?
- How the object related?
- How many objects can participate in a relationship?

An ER diagram help us answer these questions visually.

For example:

STUDENT-----ENROLLS-----COURSE

This tells us that student enroll in course.

Basic Components of an ER Diagram

The major component of an ER diagram are:

- 1) Entity
- 2) Attribute
- 3) Relationship
- 4) Key attribute
- 5) Cardinality
- 6) Participation

1. Entity: An entity is a real world object or concept about which we want to store information.

Student
Teacher
Employee
Department
Course
Customer
Product
Bank Account

For example, in a college database:

"Student" Is an entity because we want to store information about students.

Entity Representation

In a traditional ER diagram, an entity is represented by a rectangle.

Student

Entity Set: An entity set is a collection of similar entities.

For example:

"Student" is an entity type/set, while individual students are entities.

Suppose a college has:

101	Aditya
102	Deeptanarayan
103	Soumodip

Each individual student is an entity, while all students together form the **Student entity set**.

2. Attribute: An attribute describes a property or characteristics of an entity. For example, a student can have:

Student

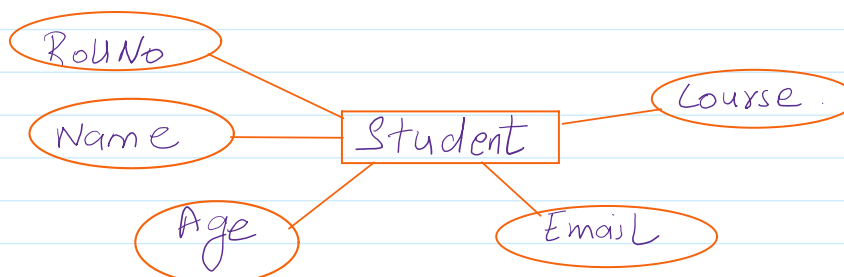
|
+--RollNo
+--Name
+--Age
+--Email
+--Course

Here:

- RollNo is an Attribute.
- Name is an Attribute.
- Age is an Attribute.
- Email is an Attribute.
- Course is an Attribute.

Representation

An attribute is traditionally represented by an Oval/ellipse.



Type of Attributes

There are several important types of attributes.

Simple Attribute

A simple attribute cannot be meaningfully divided into smaller attributes.

Example:

Age
Gender

Salary
RollNo

Composite Attribute

A composite attribute can be divided into smaller attributes.

For example:

"Name"

Can be divided into:

FirstName

MiddleName

LastName

Similarly:

"Address"

Could contain:

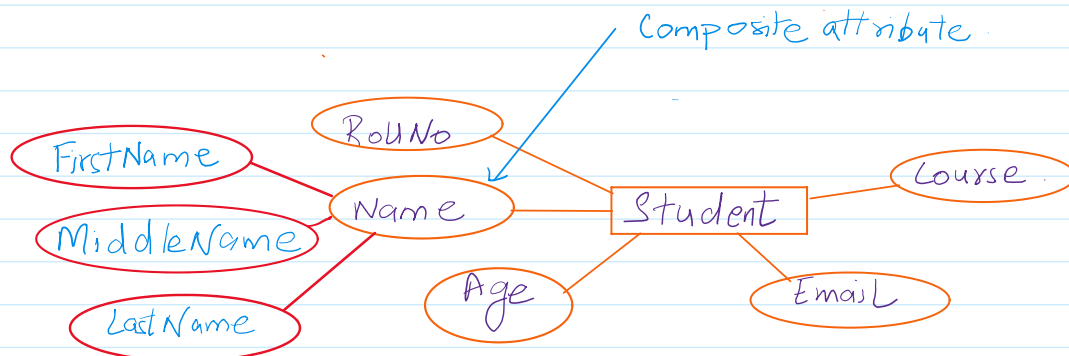
HouseNo

Street

City

State

Pincode



Single-Valued Attribute

A single - valued attribute has only one value for a particular entity.

Example:

Student -> DateOfBirth

A student normally has one date of birth.

Example:

RollNo

DateOfBirth

AdharNumber

EmployeeID

PassportNo

Multi-valued Attribute

A multi valued attribute can have multiple values for one entity.

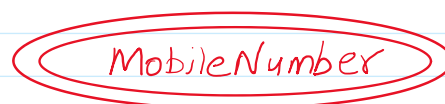
For example, an employee may have multiple mobile numbers:

Employee

|

+-9876545678

+-6345678233



In an ER diagram a multi valued attribute is represented using a double Oval.
Another example, a Student might have multiple emails.

Student

|

+-personal@gmail.com



Student

|

+---personal@gmail.com

+---college.edu@KRMangalam.edu

Email

Depending on the database requirements.

Derived Attributes

A derived attribute is an attribute whose values can be calculated from another attribute(s).

Example: DateOfBirth -> Age

If the date of birth is known, age can be calculated.

In an ER diagram, a derived attribute is represented using a dashed Oval.

Age

Another example:

(Salary + Bonus) -> totalSalary

totalSalary

Key Attributes: A key attribute uniquely identifies an entity.

For example, suppose we have:

Student

|

+--RollNo

+--Name

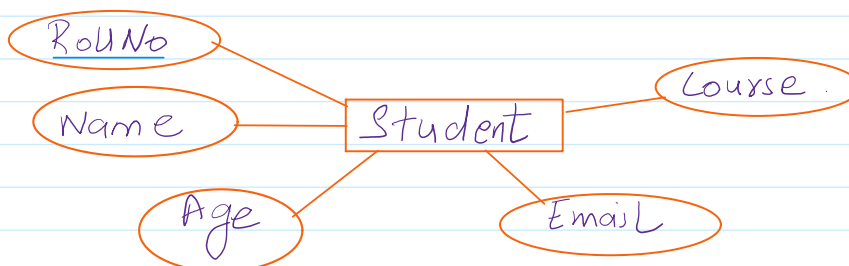
+--Age

+--Email

Two students could have the same name, but their **RollNo** should uniquely identify them. Therefore **RollNo** can be the primary key.

ER representation

A key attribute is traditionally represented by an underline attribute.



Here, 'RollNo' is underlined, Hence it is a Primary key.

Relationship

A relationship represents an association between two or more entities.

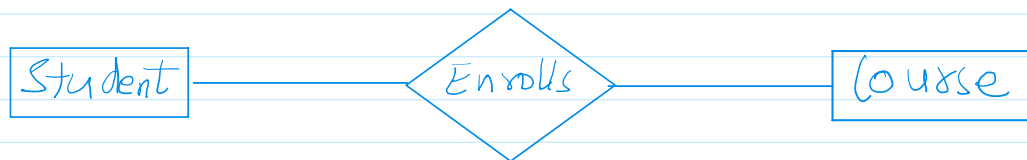
For example:

STUDENT ---- ENROLLS ----- Course

Here:

- STUDENT is an entity.
- COURSE is an entity.
- ENROLLS is a relationship.

In traditional ER diagram, a relationship is represented by a diamond.



ER relationship diagram

Relationship Name

A relationship should normally have a meaningful name.

Examples:

Entity1	Relationship	Entity2
Student	Enrolls	Course
Employee	Works_For	Department
Customer	Places	Order
Doctor	Treats	Patient
Teacher	Teaches	Subject
Customer	Purchases	Product

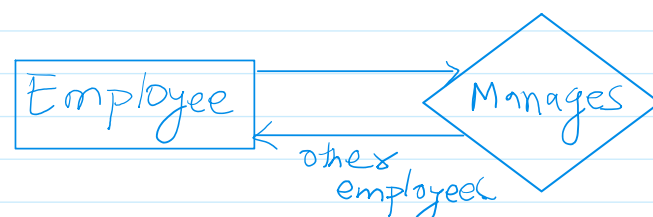
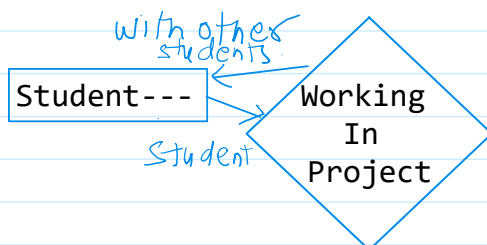
Degree of Relationship

The degree of a relationship specifies how many entity types participate in the relationship.

The major types are:

1. Unary
2. Binary
3. Ternary

1. Unary relationship: A unary relationship involves one entity type.



The same entity participate in the relationship in different roles:
Manger is an Employee managing other Employee

2. Binary Relationship: A binary relationship involves two entity types. This is the most common type.

Example:

Student --- Enrolls --- Course

Two entities participates: Student, Course
Therefore it is a binary relationship.

Other examples:

Employee --- Works_for --- Department

Customer --- Places --- Order

Doctor --- Treats --- Patient

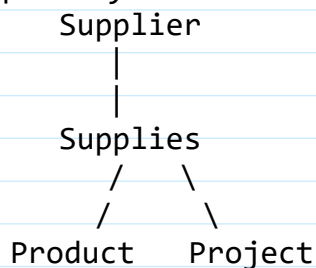
3. Ternary relationship: A ternary relationship involves three entity types. For example, consider:

Supplier, Product, Project

Suppose a supplier supplies a particular product to a particular project.

The relationship could be: Supplies

Conceptually:



Three entities participate in the relationship, so it is called a **ternary relationship**.

4. Cardinality: Cardinality describes how many entities of one entity set can be associated with entities of another entity set.

The major types are:

- i. One-to-One(1:1)
- ii. One-to-Many(1:N)
- iii. Many-to-One(N:1)
- iv. Many-to-Many(N:N)

This is one of the most important concept when drawing ER diagrams.

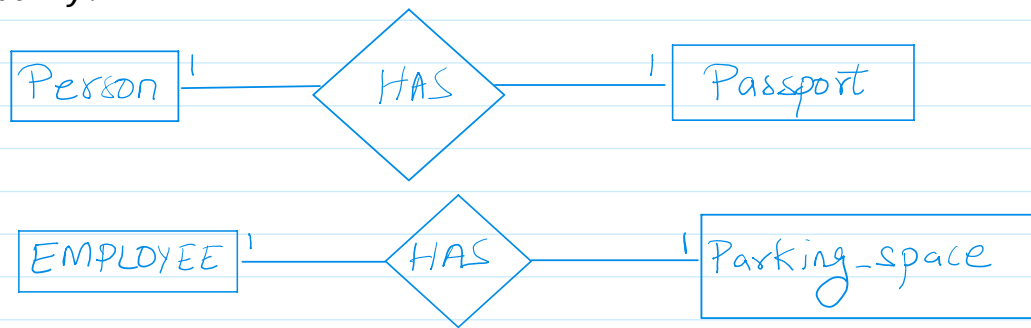
1. One-to-One Relationship(1:1)

In a **one-to-one relationship**, one entity from entity A is associated with at most one entity from entity B and vice versa.

Example:

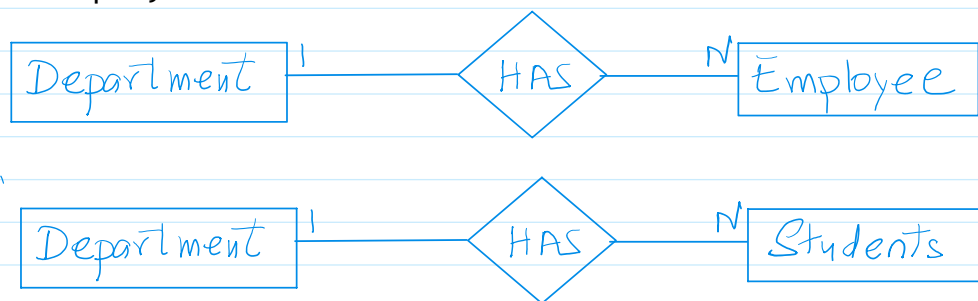
Person --- HAS --- Passport
A person has one passport and a passport belongs to one person.

Conceptually:



If each employee gets at most one parking space, and each parking space belongs to at most 1 employee, the relationship is 1: 1.

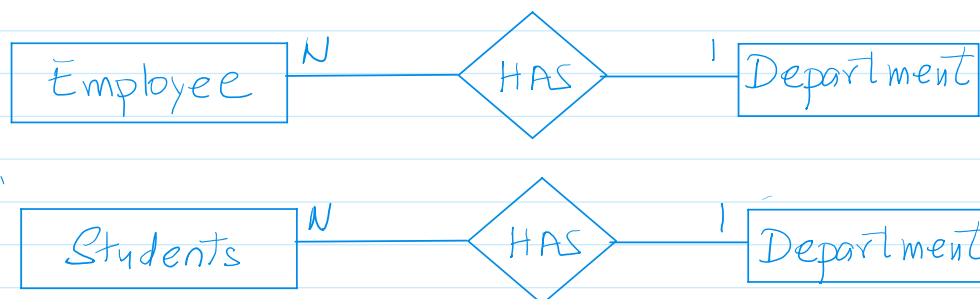
2. One-to-Many Relationship(1:N): In a one-to-many relationship, one entity from the first entity set can be associated with many entities from the 2nd entity set. For example,



One department can have many employees.
But an employee belongs to one department.

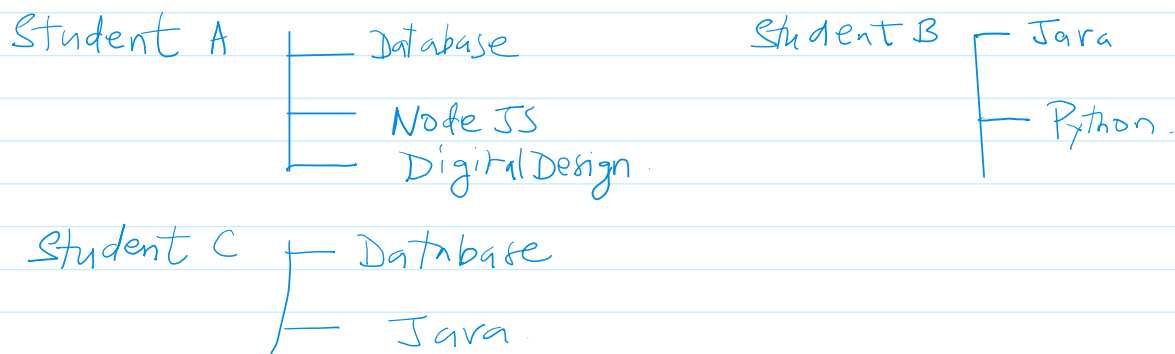
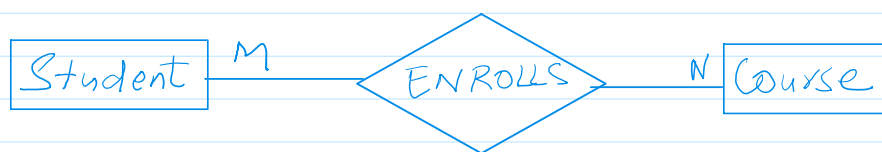
One department can have many students.
But one student belongs to one department in a college.

3. Many-to-One Relationship(N:1): This is the reverse perspective of one-to-many. If we draw the above diagram in reverse then we get many-to-one relationship.



4. Many-to-Many Relationship: In many too many relationship, many entities from one entity set can be associated with many entities from another entity set.

Example:



This is an M:N relationship

Participation Constraint

Participation Specifies whether the participation of an entity in a relationship is mandatory or optional.

There are two types:

1. **Total Participation:** Every entity must participate in the relationship. It is traditionally represented using a double line.

Example:



If every *Employee* must belong to a department, *Employee* participation may be total.

2. **Partial Participation:** Some entities may participate in the relationship, while others may not. It is traditionally represented using a single line. For example, not every employee may manage another employee.



Therefore, employee participation in managers can be partial.

Weak Entity

A **weak entity** is an entity that cannot be uniquely identified using its own attribute alone. It depends on another entity called the strong entity.

Example:

Consider:

Employee
Dependent
A dependent might have:
DependentName
Age
Relationship

A DependentName alone might not uniquely identify a dependent across all employees.

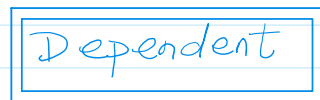
Example:

Employee 101 -> Ajay

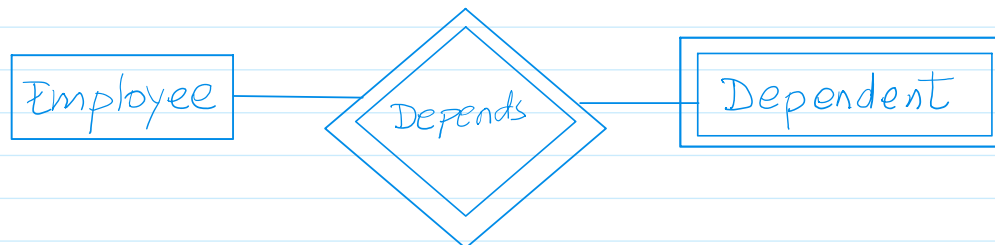
Employee 102 -> Vijay

Therefore the depend. And maybe identify using:
EmployeeID + DependentName

A weak entity is traditionally represented using a double rectangle.



It's identifying relationship is traditionally represented using a double diamond.



Strong Entity: A strong entity has its own primary key and does not depend on another entity for identification.

Example:

Student

with RollNo as Primary Key

Home work:

Student-Course Database

Let's design an ER diagram for a college.

Requirements:

A college wants to store :

- Student information
- Course information
- Student enrollment information

A student can enroll in many courses.

A course can have many students.

Step-1 : Identify Entities

Step-2 : Identify attributes

Step-3: Identify Keys

Step-4: Identify relationship

Step-5: Identify Cardinality

Step-6: Draw ER Diagram

